Exercise 3

2. Cloning

–>im richtigen Directory mit **git clone https://github.com/anuchii/konfigExercises.git**

–> Nicht in Directory aber Ordner angeben wo das repository geklont wird

```
> git clone https://github.com/anuchii/konfigExercises.git zweiteClone
Cloning into 'zweiteClone'...
remote: Enumerating objects: 35, done.
remote: Total 35 (delta 0), reused 0 (delta 0), pack-reused 35 (from 2)
Receiving objects: 100% (35/35), 61.14 MiB | 3.69 MiB/s, done.
Resolving deltas: 100% (3/3), done.
```

–> History: Alle commits werden geholt

```
* 1eb26e4 (HEAD -> main, origin/main, origin/HEAD) Add selbstchecks
* dae4cb4 Add exercises 1-2
* f26866d Add documents
* 9f478a3 Add .gitignore
* 8b61cd7 Initial commit
(END)
```

3. Add Remote

-> zuerst : git init

```
> git remote add origin https://github.com/anuchii/konfigExercises.git
> git remote -v
origin  https://github.com/anuchii/konfigExercises.git (fetch)
origin  https://github.com/anuchii/konfigExercises.git (push)

~/Desktop/FH/zweiteClone  on main ?5
>
```

4. New Files

- –> erster Push klappt

- –> Beim zweiten push ist der Stand schon veraltet, weil Remote inzwischen einen neuen commit vom ersten Repo hat.

```
> git push
To https://github.com/anuchii/konfigExercises.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/anuchii/konfigExercises.g
it'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

- –>Lösung: zuerst git pull –rebase origin main , dann git push

5. Conflict

- Git weiß nicht automatisch, welche Version richtig ist.
- In der Datei steht dann
- Lösung: Konflikt manuell lösen.
- Am Ende darf ‘<<<<<<<’, ‘=====’ und ‘>>>>>>>’ nicht mehr in der Datei stehen
- –> git add
- –> git rebase - - continue
- –> git push

```
> git pull --rebase origin main
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 7 (delta 3), reused 7 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 100% (7/7), 579 bytes | 48.00 KiB/s, done.
From https://github.com/anuchii/konfigExercises
 * branch            main       -> FETCH_HEAD
   e1eb566..3302733  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
error: could not apply d2d1905... Modify README.md option b
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue"
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git
abort".
hint: Disable this message with "git config set advice.mergeConflict fa
Could not apply d2d1905... # Modify README.md option b

~/Desktop/FH/zweiteClone  on @33027339 rebase-i 1/1 ~1
>
```

```
# Konfig-Repository
<<<<<<< HEAD
## Option A
=======
## Option B

>>>>>>> d2d1905 (Modify README.md option b)
```

6. Tagging

- Repo A:
```
> git tag  -a v1.0.0 -m "Version 1.0.0"
```
- –>Tags werden nicht automatisch gepusht:
```
> git push origin v1.0.0
```
- Repo B:
```
> git fetch --tags
```
- Überprüfen durch : git tag

7. Two Features

- Repo A

```
> git checkout -b featureA
Switched to a new branch 'featureA'

~/Desktop/FH/konfi  on featureA
>
```

```
> echo Feature A > featureA.txt
> git add featureA.txt
> fir commit -m "Add feature A"
zsh: command not found: fir
> git commit -m "Add feature A"
[featureA 3b8e3d2] Add feature A
 1 file changed, 1 insertion(+)
 create mode 100644 featureA.txt
> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
> git merge featureA
Updating 3302733..3b8e3d2
Fast-forward
 featureA.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 featureA.txt
> git push
```

- Repo B

```
> git branch featureB
> git checkout featureB
Switched to branch 'featureB'

~/Desktop/FH/zweiteClone  on featureB
```

```
> echo FeatureB > featureB.txt
> git add featureB.txt
> git commit -m "Add feature B"
[featureB 442ee62] Add feature B
 1 file changed, 1 insertion(+)
 create mode 100644 featureB.txt
```

```
> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
> git merge featureB
Updating fc427e5..6ee5269
Fast-forward
 featureB.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 featureB.txt
> git push
```

- Branches löschen
- –>git branch -d featureA

8. Major and Minor

1 Deployment-Branch erstellen + Erstes Minor Feature

```
> git checkout -b deployment
Switched to a new branch 'deployment'
> echo Minor Feature 1 > minor1.txt
> git add .
> git commit -m "Add minor feature 1"
```

```
> git tag -a v1.1.0 -m "Version 1.1.0"
> git push origin deployment
> git push origin v1.1.0
```

2 Zweites Minor Feature

```
> echo Minor Feature 2 > minor2.txt
> git add .
> git commit -m "Add minor feature 2"
> git tag -a v1.2.0 -m "Version 1.2.0"
```

```
> git push origin deployment
> git push origin v1.2.0
```

3 Neue Major-Version in Main erstellen

```
> git checkout main
> git pull --rebase origin main
> git merge deployment
> git tag -a v2.0.0 -m "Version 2.0.0"
> git push
> git push origin v2.0.0
```

4 Zweites Repo synchronisieren

```
> git checkout main
> git pull --rebase origin main
> git fetch --tags
> git checkout deployment
> git pull --rebase origin deployment
```